Actively Secure and CPA^{D}-Resistant Threshold FHE via Homomorphic Quotient Proofs

Anonymous Authors

***Abstract*—The Threshold Fully Homomorphic Encryption (ThFHE) technique mitigates single points of failure and power concentration in traditional single-key systems by distributing keys and enabling joint decryption. However, in decentralized networks with malicious nodes, ThFHE faces several challenges, including reliance on semi-honest security models, vulnerability to key leakage via chosen-plaintext attacks with decryption oracles (CPA^D), and Zero-Knowledge Proof (ZKP) bottlenecks for large ciphertext moduli. To address these issues, we propose a ThFHE scheme that provides active security and circuit privacy. The proposed scheme uses a compact RNS-GLWE architecture, floods noise in decentralized Relinearization Key Generation (RLKGen) and partial decryption for dynamic, decentralized threshold structures, and employs a Homomorphic Quotient Fiat-Shamir Zero-Knowledge Proof (HQ-FS ZKP) mechanism that confines proof generation to a small baseline modulus and uses a Breadth-First Search (BFS) queue for batch filtering. We further design a zero-ciphertext superposition mechanism to resist CPA^D attacks and achieve FuncCPA security. By introducing variance-flooding noise at the terminal stage of homomorphic operations, our scheme truncates algebraic correlations and computational traces in ciphertexts without relying on resource-intensive bootstrapping. Performance evaluations show that the scheme requires 1.176 ms per node to generate a proof and yields an active-defense latency of 642.76 ms. Compared with state-of-the-art schemes with equivalent security, these metrics are reduced by 31.6% and 72.1%, respectively, improving active security validation latency and resistance to key-recovery attacks while maintaining architectural compactness.**

***Index Terms*—Distributed Systems Security, Threshold Fully Homomorphic Encryption, Active Security, CPA^{D} Security, Circuit Privacy, Zero-Knowledge Proofs.**

I. Introduction

With the rapid development of collaborative computing architectures such as cloud computing, federated learning, and cross-institutional data sharing, performing collaborative analysis while maintaining data locality has become a key challenge in contemporary information security. Fully homomorphic encryption (FHE) can compute arbitrary-depth functions on encrypted data, offering an ideal cryptographic solution to this problem. However, traditional single-key FHE systems suffer from single points of failure and excessive power concentration, making them unsuitable for decentralized applications where participants do not trust each other. Meanwhile, multi-key FHE systems face an exponential increase in computational load and noise due to interactions between different keys. To address these issues, Threshold Fully Homomorphic Encryption (ThFHE) has emerged. ThFHE distributes decryption authority and trust across multiple network nodes through decentralized key generation and collaborative decryption protocols, while using a single public key to significantly improve computational efficiency over multi-key FHE. While Threshold Fully Homomorphic Encryption (ThFHE) holds immense promise for privacy-preserving distributed computing, malicious actors obstruct its transition into real-world, decentralized environments. Bridging the gap between theoretical security and practical deployment drives this research, targeting critical vulnerabilities and performance barriers:

- *Exposed to Active Malicious Attacks and Key Leakage:* Most existing ThFHE protocols assume semi-honest adversaries, which is unsuitable in untrusted networks. Active adversaries who deliberately insert malformed or malicious decryption shares during collaborative decryption can do more than corrupt computations. They can exploit such deviations to perform sophisticated key-recovery attacks, namely Chosen-Plaintext Attacks with Decryption oracles (CPA^D), which can reveal the underlying global secret key and cause a total breakdown of system security.
- *Performance Bottlenecks from Verifiability:* Conventional cryptosystems against active foes use Zero-Knowledge Proofs (ZKPs) to enforce honest node behavior. However, traditional FHE schemes operate over an extraordinarily large ciphertext modulus $\mathcal{Q}$. Forcing ZKP generation and verification to execute directly over this massive algebraic space imposes exorbitant computational loads and excessive communication bandwidth requirements. This overhead makes conventional ZKPs-based verification too resource-intensive for practical deployment.
- *Circuit Confidentiality and FuncCPA Protection:* Multi-party homomorphic evaluation and distributed relinearization key (RLK) generation require algebraic operations that create predictable noise patterns and secret-key cross-term indicators in ciphertext. If a protocol fails to mask or remove these residual algebraic characteristics, attackers can analyze computational traces to reverse-engineer the evaluation circuit's internal state. Such leakage violates Function-Privacy Chosen-Plaintext Attack (FuncCPA) security, leaving the system vulnerable to algebraic tracing.

A. Related Work

1) Threshold and Multi-Party FHE: Asharov et al. [1] used threshold FHE for multiparty computation with few interaction rounds, and Boneh et al. [2] showed that threshold FHE can serve as a universal thresholdizer for other cryptographic primitives. Besides, Mouchet et al. [3] designed RLWE-based multiparty BFV variant, yielding practical N-out-of-N protocols. Recent works have focused on reducing the cost of

threshold decryption. Dahl et al. [4] showed that noise flooding remains feasible to small-parameter schemes (e.g., ThFHE) by switching to larger parameters before decryption. Chowdhury et al. [5] proposed a threshold FHE scheme with a polynomial modulus-to-noise ratio, based on a Rényi-divergence analysis of threshold decryption, and implemented it on Torus FHE. Boudgoust and Scholl [6] achieved threshold encryption with only a polynomial modulus; however, Passelègue and Stehlé [7] found it vulnerable and subsequently devised a scheme with small ciphertexts and decryption shares. Chang and Li [8] lowered the overhead of supporting arbitrary thresholds, with experiments involving a thousand parties, and Kim et al. [9] further reduced the modulus required by asynchronous Shamir-based schemes. Their construction also reduced the storage required at each party under strong simulation security.

2) CPA^D Attacks and FuncCPA Security: Li and Micciancio [10] demonstrated that IND-CPA security does not capture the security of approximate HE under decryption queries. Their attack uses decryption access to honestly evaluated ciphertexts to recover the CKKS secret key, motivating the IND-CPA^D security notion. Likewise, subsequent work showed that related attacks arise in exact HE schemes. Cheon et al. [11] exploited the non-negligible decryption failure rate of practical BFV and TFHE parameters to mount distinguishing and key-recovery attacks, which extend to threshold variants whose correctness is similarly loose. Checri et al. [12] gave practical key-recovery attacks on BFV, BGV, and TFHE, and observed that threshold versions are especially exposed when no extra noise protects decryption shares. Further, Bendoukha et al. [13] confirmed that such attacks are realistic in federated learning built on threshold FHE. Akavia et al. [14] studied a related gap in client-aided outsourcing in which a malicious server receives refreshed ciphertexts from the client. To formalize this setting, they introduced funcCPA, a notion stronger than CPA but weaker than CCA2. They showed that HE schemes meeting appropriate circuit-privacy requirements, including schemes that support ciphertext sanitization [15] or achieve circuit privacy through refined noise analysis [16], can achieve this notion.

3) Malicious Security and Zero-Knowledge Proofs: In multiparty FHE, malicious parties may send malformed keys or protocol messages. For malicious security, parties attach ZKPs to keys and protocol messages to establish validity without revealing secret information. Lattice-based zero-knowledge techniques, ranging from Fiat–Shamir with aborts [17] to recent compact proof systems [18], can construct such proofs. PELTA [19] combines lattice-based commitments and proof systems to check multiparty FHE operations in zero knowledge, and Hwang et al. [20] built knowledge-sound zero-knowledge arguments for multiparty HE on top of polynomial interactive oracle proofs, reducing proof size and verification time. Xie et al. [21] used commitments and NIZK proofs to provide active security for a Shamir-based threshold scheme. Chen et al. [22] built a fully decentralized ThFHE on a full-RNS BFV architecture, combining distributed key generation and MPC-based relinearization key generation with NIZK proofs and smudging noise, so that each node's partial decryption can be publicly verified. Zyskind et al. [23] designed a protocol that uses preprocessing and MPC-based rounding to remove noise, thereby avoiding noise flooding and achieving universally composable threshold decryption.

B. Motivation and Contributions

As ThFHE becomes increasingly important for decentralized, privacy-preserving computation, balancing robust protection against sophisticated attacks under malicious models with system efficiency remains a major challenge. To address limitations in current studies, this research aims to secure systems against practical CPA^D attacks and achieve rigorous funcCPA security without the high computational costs of traditional sanitization or bootstrapping. Concurrently, we aim to reduce the computational and communication overhead of running full-fledged ZKPs over enormous ciphertext moduli. Finally, we aim to resolve the difficulty of combining arbitrary threshold flexibility with ciphertext compactness in RNS architectures, ensuring that dynamic, decentralized threshold structures can be supported without sacrificing computational or communication performance. To address these critical issues in malicious decentralized environments, we propose an actively secure, circuit-privacy-preserving Threshold FHE system. The primary contributions of the proposed work are as follows:

- Our system features a compact RNS-GLWE architecture that securely integrates massive flooding noise directly into Relinearization Key Generation (RLKGen) and partial decryption, ensuring efficient support for dynamic decentralized threshold structures.
- We propose the Homomorphic Quotient Fiat–Shamir (HQ-FS) ZKP mechanism, which restricts proof generation to a small baseline modulus $K_{\mathcal{Q}}$. Integrated with a BFS batch-filtering queue, this design drastically reduces verification overhead while guaranteeing active security and precise isolation of malicious nodes.
- We introduce a novel *zero-ciphertext superposition* mechanism that injects high-variance flooding noise immediately following homomorphic evaluations. This effectively truncates algebraic dependencies and computational traces in ciphertexts, defending against practical $\mathrm{CPA^D}$ attacks without requiring computationally expensive bootstrapping or traditional cleanup operations.
- By incorporating zero-ciphertext superposition into the security framework, we formally prove that our scheme achieves IND-FuncCPA security and circuit privacy, resisting advanced key-recovery attacks under up to $N-1$ active colluding adversaries.
- The performance analysis shows that, compared with related actively secure threshold FHE systems, the proposed work reduces proof-generation latency and active-defense overhead for large-scale decentralized deployments.

The remaining paper is organized as follows: Section II provides essential preliminaries. Section III discusses the proposed protocol in detail. Sections IV and V discusses the

security and performance analyses of the proposed protocol. Finally, Section VI concludes this work.

II. Preliminaries

This section introduces the theoretical prerequisites and summarizes key notations in Table I.

A. General Learning With Errors Assumption

Let $\mathcal{R}_{\mathcal{Q}} = \mathbb{Z}_{\mathcal{Q}}[x]/(x^d+1)$ and χ_η be an error distribution with coefficients bounded by η. General Learning With Errors (GLWE) uses n-dimensional vectors over a ring of degree d. It includes LWE (d=1) and RLWE (n=1) as special cases [24].

Definition 1 (Decisional GLWE Assumption)**.** *Let $\mathbf{A}\in\mathcal{R}_{\mathcal{Q}}^{m\times n}$ be uniformly random, with $\mathbf{s}\leftarrow\chi_\eta^n$ and $\mathbf{e}\leftarrow\chi_\eta^m$. The decisional GLWE assumption states that, for every PPT adversary $\mathcal{A}$, we have*

$$\left|\Pr[\mathcal{A}(\mathbf{A},\mathbf{As}+\mathbf{e})=1]-\Pr[\mathcal{A}(\mathbf{A},\mathbf{b}^*)=1]\right|\le \mathsf{negl}(\lambda) \quad (1)$$

where $\mathbf{b}^*$ is sampled uniformly from $\mathcal{R}_{\mathcal{Q}}^m$ [5]. We write this advantage as $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{GLWE}}(\lambda)$. Concrete parameters follow [24].

In GLWE-based encryption [22], a plaintext $m\in\mathcal{R}_T = \mathbb{Z}_T[x]/(x^d+1)$ is encrypted as

$$c=(\mathbf{u},v)=\left(\mathbf{A}^\top\mathbf{r}+\mathbf{y}_0,\ \langle\mathbf{pk},\mathbf{r}\rangle+y_1+\Delta m\right) \mod \mathcal{Q} \quad (2)$$

where the public key $\mathbf{pk}=\mathbf{As}+\mathbf{e} \pmod{\mathcal{Q}}$, $\mathbf{r}\leftarrow\chi_\eta^m$, $\mathbf{y}_0$ and y_1 are error terms. These error terms are drawn from the flooding distribution (see Algorithm 4). The message is multiplied by $\Delta=\lfloor\mathcal{Q}/T\rceil$ so that it sits in the high-order part of each coefficient. For decrypt, we compute $v-\langle\mathbf{u},\mathbf{s}\rangle=\Delta m+e_{dec} \pmod{\mathcal{Q}}$, then divide by Δ and round. This recovers m whenever $\|e_{dec}\|_\infty<\Delta/2$.

B. Residue Number System

In our scheme, the ciphertext modulus $\mathcal{Q}$ is too large to fit in a single machine word. Arithmetic on such large integers can bottleneck lattice-based HE. A Residue Number System (RNS) represents an element by its residues modulo several small, word-sized primes. Ring operations can then be performed independently modulo each prime.

Definition 2 (Residue Number System)**.** *Let $Q=\{q_1,\dots,q_r\}$ be a set of distinct prime moduli, and let $\mathcal{Q}=\prod_{i=1}^r q_i$ be the full composite modulus. By the Chinese Remainder Theorem (CRT), there exists a ring isomorphism:*

$$\mathcal{R}_{\mathcal{Q}}\cong\prod_{i=1}^r\mathcal{R}_{q_i}=\mathcal{R}_{q_1}\times\cdots\times\mathcal{R}_{q_r} \quad (3)$$

using the element-wise mapping $a(x)\mapsto(a_1(x),\dots,a_r(x))$, where $a_i(x):=a(x) \bmod q_i\in\mathcal{R}_{q_i}$ for each prime $q_i\in Q$. Under this representation, component-wise operations satisfy:

$$(a\circ b)_i\equiv a_i\circ b_i \pmod{q_i}, \quad \textit{where } \circ\in\{+,\cdot\} \quad (4)$$

Homomorphic operations are evaluated independently modulo each small prime q_i without computing directly modulo the full integer $\mathcal{Q}$. CRT reconstruction is deferred to the final stage of decryption (see Algorithm 6), where $a(x)\in\mathcal{R}_{\mathcal{Q}}$ is rebuilt from its residues and centered within $(-\mathcal{Q}/2,\mathcal{Q}/2]$. The CRT representation supports the RNS gadget decomposition used in relinearization. Let $\hat{q}_i=\mathcal{Q}/q_i$. For any $w\in\mathcal{R}_{\mathcal{Q}}$,

$$w\equiv\sum_{i=1}^r\left[w\cdot\hat{q}_i^{-1}\right]_{q_i}\cdot\hat{q}_i \pmod{\mathcal{Q}} \quad (5)$$

where $[\cdot]_{q_i}$ is the centered reduction modulo q_i, so the coefficients of each digit are bounded by $q_i/2$ in absolute value. During relinearization, we multiply these digits by the key material encrypting $\mathbf{s}_a\mathbf{s}_b\hat{q}_i$. Since each digit is much smaller than w, the noise introduced in this step remains small. Since each q_i divides $\mathcal{Q}$, reduction from $\mathcal{R}_{\mathcal{Q}}$ to $\mathcal{R}_{q_1}$ preserves ring operations, so a linear relation that holds modulo $\mathcal{Q}$ also holds modulo q_1. We use this fact to run the zero-knowledge checks on the single limb q_1 instead of the full modulus. We call q_1 the baseline modulus also write $K_{\mathcal{Q}}=q_1$. Working modulo $K_{\mathcal{Q}}$ is what keeps the proof computation in HQ-FS cheap. The primary basis Q is not always enough, since some intermediate values in homomorphic evaluation exceed $\mathcal{Q}$. Algorithm 1 therefore also fixes an auxiliary basis $P=\{p_1,\dots,p_k\}$ of primes coprime to $\mathcal{Q}$, with $\mathcal{P}=\prod_{j=1}^k p_j$. Base extension moves residues from Q to $Q\cup P$, so no full CRT reconstruction is needed. With fast base extension, the output can be off by a small multiple of $\mathcal{Q}$, which we treat as part of the noise.

TABLE I
List of Notations

Symbol	Description
λ	Security parameter
d	Degree of polynomial
(m,n)	Dimension of Matrix
T	Plaintext modulo
B_{flood}	funcCPA flood noise bound
B_{zkp}	ZKP noise bound
N	Total number of nodes in the decentralized network
t	The global reconstruction threshold during the DKG phase
N_c	The size of the dynamically selected decyrption committee
t_c	The internal decryption threshold within the active committee
$\mathcal{Q}$	The primary ciphertext modulus
$\mathcal{P}$	The auxiliary extended modulus
K	The quotient modulus
$\mathcal{R}_{\mathcal{Q}},\mathcal{R}_{\mathcal{P}},\mathcal{R}_K$	The base of polynomial ring
Δ	Scaling factor
χ	Error distribution
$\mathcal{U}$	Uniform distribution
$\mathbf{A}$	Global encrypt matrix
$\mathbf{A}_{zkp}$	HQ-FS ZKP public reference matrix
$\mathcal{L}$	Node set

C. Zero-Knowledge Proofs

A malicious node may send an incorrect value as its partial decryption share. Without an additional check, the requester cannot determine whether the share was computed correctly. Each node therefore provides a proof with its share, allowing the requester to verify the computation without learning the secret information used to produce it.

Definition 3 (Non-Interactive Zero-Knowledge Argument)**.** *Let $\mathcal{R}\subseteq\mathcal{X}\times\mathcal{W}$ be an NP relation with associated language*

$\mathcal{L}_{\mathcal{R}} = \{x \mid \exists w \text{ s.t. } (x,w) \in \mathcal{R}\}$. *A Non-Interactive Zero-Knowledge (NIZK) argument system for $\mathcal{R}$ consists of a tuple of PPT algorithms* $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ *satisfying:*

- ***Completeness:*** *For any $(x,w) \in \mathcal{R}$ and* $\mathrm{crs} \leftarrow \mathsf{Setup}(1^\lambda)$, *an honest prover generating* $\pi \leftarrow \mathsf{Prove}(\mathrm{crs}, x, w)$ *satisfies* $\mathsf{Verify}(\mathrm{crs}, x, \pi) = 1$.
- ***Computational Soundness:*** *For any $x \notin \mathcal{L}_{\mathcal{R}}$ and any PPT adversary $\mathcal{A}$, when* $\mathrm{crs} \leftarrow \mathsf{Setup}(1^\lambda)$, *we have:* $\Pr[\mathsf{Verify}(\mathrm{crs}, x, \pi) = 1 \mid \pi \leftarrow \mathcal{A}(\mathrm{crs})] \le \mathsf{negl}(\lambda)$.
- ***Zero-Knowledge:*** *There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that $\forall (x,w) \in \mathcal{R}$, real and simulated proofs are computationally indistinguishable, i.e.,* $\mathsf{Prove}(\mathrm{crs}, x, w) \approx_c \mathcal{S}_2(\mathrm{crs}, \tau, x)$. *In the Random Oracle Model (ROM), $\mathcal{S}$ may program oracle responses.*

Definition 4 (Σ-Protocol)**.** *The Σ for an NP relation $\mathcal{R}$ is a three-move interactive proof system between a prover and a verifier. The prover sends a commitment $\mathbf{T}$, the verifier returns an uniform random challenge ch from a challenge space $\mathcal{C}_{ch}$, and the prover responds with $\mathbf{z}$. The verifier accepts or rejects based on the transcript $(x, \mathbf{T}, ch, \mathbf{z})$. The protocol satisfies:*

- ***Completeness:*** *If $(x,w) \in \mathcal{R}$, an honest prover for witness w returns a transcript that the verifier always accepts.*
- ***Special Soundness:*** *Given two valid transcripts $(\mathbf{T}, ch, \mathbf{z})$ and $(\mathbf{T}, ch', \mathbf{z}')$ for the same commitment $\mathbf{T}$ with $ch \neq ch'$, there exists a PPT extractor that efficiently computes a valid witness w such that $(x,w) \in \mathcal{R}$.*
- ***Special Honest-Verifier Zero-Knowledge (SHVZK):*** *There exists a PPT simulator that, given x and any challenge $ch \in \mathcal{C}_{ch}$, outputs a commitment $\mathbf{T}$ and response $\mathbf{z}$ such that the simulated transcript $(\mathbf{T}, ch, \mathbf{z})$ is statistically (or computationally) indistinguishable from an honest protocol transcript.*

Definition 5 (Fiat-Shamir Heuristic)**.** *The Fiat-Shamir transform removes the verifier's message in a Σ-protocol by deriving the challenge from the statement and commitment: $ch = H(x, \mathbf{T})$. The hash function H is modeled as a random oracle, and the resulting proof is $\pi = (\mathbf{T}, ch, \mathbf{z})$. If the underlying Σ-protocol satisfies completeness, special soundness, and SHVZK, the resulting NIZK argument complete, computationally sound, and zero-knowledge in the ROM. For a cheating prover making at most q_H queries to H, the soundness error is bounded by $\mathcal{O}(q_H/|\mathcal{C}_{ch}|)$ [22].*

Lemma 1 (Smudging)**.** *Let $x \in \mathbb{Z}$ satisfy $|x| \le B_{sec}$, and sample e_{flood} uniformly from $[-B_{flood}, B_{flood}] \cap \mathbb{Z}$. The statistical distance between e_{flood} and $x + e_{flood}$ is at most B_{sec}/B_{flood}. For an element in $\mathcal{R}_{\mathcal{Q}}$ of degree d, applying this bound coefficient-wise gives a distance of at most $d \cdot B_{sec}/B_{flood}$. Thus, choosing $B_{flood} \ge 2^\lambda d \cdot B_{sec}$ bounds the distance by $2^{-\lambda}$, making $x + e_{flood}$ statistically indistinguishable from e_{flood}.*

Proof. Consider a single coefficient first. Here e_{flood} is uniform on $[-B_{flood}, B_{flood}] \cap \mathbb{Z}$, and $x + e_{flood}$ is uniform on the same set shifted by x, so both supports contain $2B_{flood} + 1$ integers. The two supports share all but $|x|$ of them. The statistical distance is therefore $|x|/(2B_{flood}+1) \le B_{sec}/B_{flood}$. For an element of $\mathcal{R}_{\mathcal{Q}}$, we apply this to each of the d coefficients and add up the distances, which gives $d \cdot B_{sec}/B_{flood}$. ■

D. Hash Function

A cryptographic hash function is a deterministic map $H : \{0,1\}^* \to \{0,1\}^{\ell_H}$ that maps an arbitrary-length bitstring to a fixed-length digest. Three standard properties are considered. *Preimage resistance* means that, given a digest y, it is computationally infeasible to find m such that $H(m) = y$. *Second-preimage resistance* means that, for a given input m, it is infeasible to find another input $m' \neq m$ with $H(m') = H(m)$. *Collision resistance* means that it is infeasible to find any distinct inputs $m \neq m'$ such that $H(m) = H(m')$. In the HQ-FS proof, H derives the Fiat–Shamir challenge and is modeled as a random oracle $H : \{0,1\}^* \to \mathcal{C}_{\mathrm{ch}}$, where $|\mathcal{C}_{\mathrm{ch}}| = 2^d$ and

$$\mathcal{C}_{\mathrm{ch}} = \left\{ \sum_{j=0}^{d-1} c_j x^j \;\middle|\; c_j \in \{0,1\} \right\} \subset \mathcal{R}_{\mathcal{Q}}. \tag{6}$$

where x denotes the polynomial indeterminate over $\mathcal{R}_{\mathcal{Q}}$.

E. Security Definition

We consider a PPT malicious adversary $\mathcal{A}$ that statically corrupts a subset of nodes $\mathcal{L}^* \subset \mathcal{L}$ with $|\mathcal{L}^*| \le N-1$ [22]. Following [14], the security model captures potential leakage arising from homomorphic evaluations, honest nodes' partial decryption shares, and fresh ciphertexts. When $N = 1$ and the oracles defined below are omitted, the game reduces to standard IND-CPA security [5].

Definition 6 (The IND-FuncCPA Game)**.** *This game played between a challenger $\mathcal{C}$ and $\mathcal{A}$ proceeds as follows:*

1) ***Setup:*** *$\mathcal{A}$ chooses a set $\mathcal{L}^*$ of at most $N-1$ corrupted nodes. $\mathcal{C}$ simulates the remaining honest nodes. $\mathcal{C}$ and $\mathcal{A}$ jointly run key generation and relinearization-key generation, after which $\mathcal{C}$ publishes $\mathbf{pk}_{global}$.*
2) ***Phase 1:*** *$\mathcal{A}$ may adaptively query two oracles:*
   - **Evaluation Oracle:** *For a circuit f and ciphertexts, $\mathcal{C}$ returns the homomorphically evaluated ciphertext.*
   - **Partial Decryption Oracle:** *Given a ciphertext, $\mathcal{C}$ returns the honest nodes' partial decryption shares and their valid proofs.*
3) ***Challenge:*** *$\mathcal{A}$ submits equal length messages (m_0, m_1). For random $b \leftarrow \{0,1\}$ $\mathcal{C}$ sends $ct^* \leftarrow \mathsf{Enc}(\mathbf{pk}_{global}, m_b)$.*
4) ***Phase 2:*** *$\mathcal{A}$ continues to query the oracles. For a ciphertext obtained by evaluating a circuit f on ct^*, the partial decryption oracle responds only if $f(m_0) = f(m_1)$.*
5) ***Guess:*** *$\mathcal{A}$ outputs $b' \in \{0,1\}$ and wins if $b' = b$.*

The advantage of $\mathcal{A}$ is

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{IND\text{-}FuncCPA}} = \left| \Pr[b' = b] - \tfrac{1}{2} \right| \tag{7}$$

The scheme is IND-FuncCPA secure if $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{IND\text{-}FuncCPA}}$ is negligible for every PPT adversary $\mathcal{A}$.

Since each honest share includes fresh flooding noise, a valid share for ciphertext component $\mathbf{u}$ has the form $\langle \mathbf{u}, \mathbf{s}^{(k)} \rangle +$

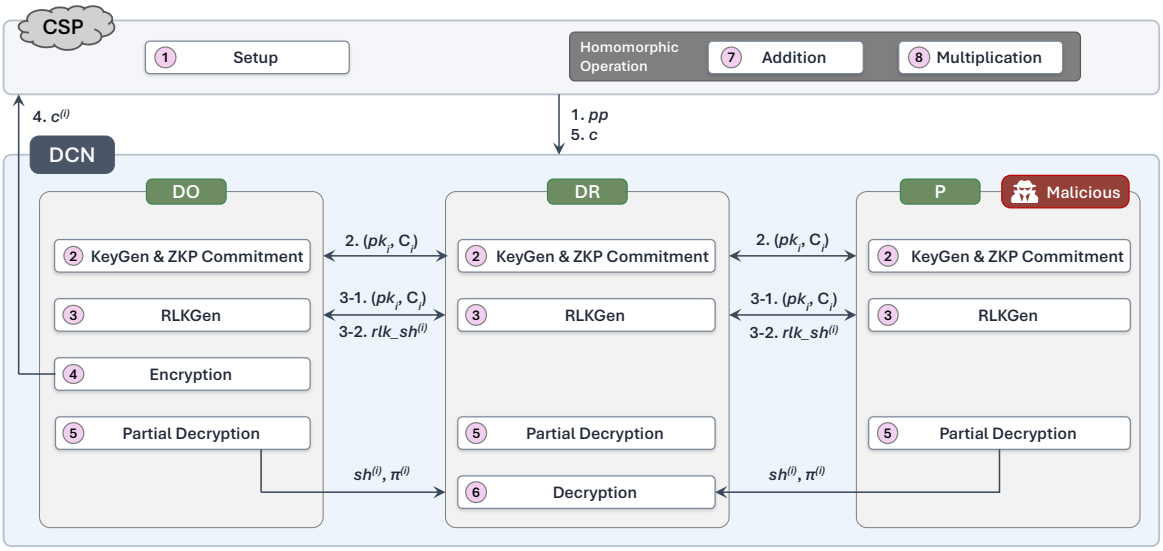


Fig. 1. System Model of the Proposed Scheme

e, where $\|e\|_\infty \leq B_{flood}$ and $\mathbf{s}^{(k)}$ is the secret-key share bound to commitment $\mathbf{C}^{(k)}$.

Definition 7 (Robustness [22])**.** *The scheme is robust if every PPT adversary has only negligible probability of producing, for a corrupted node $k \in \mathcal{L}^*$, an invalid share $\widetilde{sh}^{(k)}$ together with a proof $\widetilde{\pi}^{(k)}$ that passes verification:*

$$\mathsf{Verify}(\mathbf{u}, \widetilde{sh}^{(k)}, \mathbf{C}^{(k)}, \mathbf{A}_{zkp}, \widetilde{\pi}^{(k)}) = 1. \tag{8}$$

III. The Proposed Scheme

This section provides the system model and the proposed protocol in detail.

A. System Model

Fig. 1 shows a decentralized, threshold, multi-party, fully encrypted architecture consisting of a Cloud Service Provider (CSP) and a Decentralized Computing Network (DCN). The CSP is responsible for the global system Setup and for executing homomorphic addition and multiplication operations on encrypted data. The DCN comprises Data Owners (DO), a Data Receiver (DR), and generic Participants (G), explicitly accounting for malicious nodes in the threat model. All DCN nodes collaboratively participate in KeyGen, ZKP Commitment, and RLKGen, broadcasting their public key shares with commitments (pk_i, C_i) and relinearization key shares $rlk_sh^{(i)}$ across the network. The DO exclusively handles the initial data Encryption. During decryption, each node performs Partial Decryption and submits its decryption share and a ZKP, denoted as $sh^{(i)}, \pi^{(i)}$, to DR. Finally, the DR aggregates all successfully verified shares to execute the final Decryption.

B. Our Construction

Our scheme comprises various phases mentioned below.

1) **Setup:** Algorithm 1 is executed by CSP, which generates the global public parameters pp required for the entire system. It first builds the RNS by decomposing the massive ciphertext-space modulus Q and the auxiliary modulus P into a product of multiple prime bases and computing the corresponding CRT constants $\hat{q}_i$ to support subsequent efficient large-number computations. Simultaneously, it defines the plaintext space modulus T and the scaling factor Δ used for plaintext encoding. Next, the algorithm establishes the underlying polynomial quotient ring $\mathcal{R}_\mathcal{Q}$ and the uniform distribution χ_η for short noise, and specifically extracts the first prime q_1 to serve as the dedicated baseline modulus $K_\mathcal{Q}$ for executing the HQ-FS ZKP. Finally, the system randomly samples the global matrix $\mathbf{A}$ used for FHE encryption and the public reference matrix $\mathbf{A}_{zkp}$ dedicated to ZKP verification. These core components are then packaged into the public parameters pp and broadcast to DCN, laying the foundation for the subsequent distributed key generation, homomorphic operations, and threshold decryption.

2) **KeyGen and ZKP Commitment:** Algorithm 2 is executed by DCN to generate distributed keys and cryptographic commitments ensuring future decryption security. First, each node randomly samples its individual secret key vector $\mathbf{s}$ and an error vector $\mathbf{e}$ from the uniform short noise distribution based on pp, and computes its personal public key share $\mathbf{pk}$. To prove honest computation during the subsequent threshold decryption phase, the node samples an additional dedicated ZKP error vector $\mathbf{e}_{zkp}$. Then computes a public commitment $\mathbf{C}$ using the ZKP reference matrix $\mathbf{A}_{zkp}$, effectively binding the exact same secret key $\mathbf{s}$ without revealing it. Finally, the system collects the individual public key shares and commitments from all N nodes and aggregates the personal public key shares by summing them modulo $\mathcal{Q}$ to construct the shared global public key $\mathbf{pk}_{global}$.

Algorithm 1 Setup $(1^\lambda, n, m, d, B_{flood})$

Require: Security parameter λ, dimension of matrix (n, m), polynomial degree d, funcCPA boundary of noise B_{flood}
Ensure: Public parameter pp

Set ciphertext modulus basis set $Q = \{q_1, q_2, \ldots, q_r\}$
Set auxiliary modulus basis set $P = \{p_1, \ldots, p_k\}$
Compute ciphertext modulu $\mathcal{Q} = \prod_{i=1}^{r} q_i$
Define $\hat{Q} = \{\hat{q}_1, \ldots, \hat{q}_r\}$, where $\hat{q}_i = \frac{\mathcal{Q}}{q_i}$
Compute auxiliary modulu $\mathcal{P} = \prod_{j=1}^{k} p_j$
Define plaintext modulu T
Compute scaling factor $\Delta = \lfloor \frac{\mathcal{Q}}{T} \rceil$
Define polynomial ring $\mathcal{R}_\mathcal{Q} = \mathbb{Z}_\mathcal{Q}/[x^d + 1]$
Define the uniform distribution χ_η
Set the ZKP verification baseline prime $K_\mathcal{Q} = q_1$
Sample global encrypt matrix $\mathbf{A} \in \mathcal{R}_\mathcal{Q}^{m \times n}$
Sample HQ-FS ZKP public reference matrix $\mathbf{A}_{zkp} \in \mathcal{R}_\mathcal{Q}^{m \times n}$
Send public parameter $pp = \{\mathbf{A}, \mathbf{A}_{zkp}, \eta\}$ to each node

Algorithm 2 KeyGen and ZKP commitment (pp)

Require: Public parameter pp
Ensure: Public key $\mathbf{pk}$, commitment $\mathbf{C}$, global public key $\mathbf{pk}_{global}$

Sample secret vector $\mathbf{s} \in \mathcal{R}_{\chi_\eta}^n$
Sample error vector $\mathbf{e} \in \mathcal{R}_{\chi_\eta}^m$
Compute $\mathbf{pk} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{\mathcal{Q}}$
Sample ZKP error vector $\mathbf{e}_{zkp} \in \chi_\eta^m$
Compute public commitment $\mathbf{C} = \mathbf{A}_{zkp} \cdot \mathbf{s} + \mathbf{e}_{zkp} \pmod{\mathcal{Q}}$
Collect pk-Commitment set $\{(\mathbf{pk}_1, \mathbf{C}_1), (\mathbf{pk}_2, \mathbf{C}_2), \ldots, (\mathbf{pk}_N, \mathbf{C}_N)\}$
Compute global public key $\mathbf{pk}_{global} = \sum_{i=1}^{N} \mathbf{pk}_i \pmod{\mathcal{Q}}$

Algorithm 3 RLKGen $(pp, \mathbf{pk}_{global})$

Require: Public parameter pp, global public key $\mathbf{pk}_{global}$
Ensure: Global relinearization key RLK_{global}

for $a \in \{1, \ldots, n\}$ **do**
  Sample random polynomial vector $\mathbf{r}_a \in \mathcal{R}_{\chi_\eta}^m$
  Sample flood error vector $\mathbf{e}_{u,a} \in \mathcal{R}_{\chi_{flood}}^n$
  Sample flood error polynomial $e_{v,a} \in \mathcal{R}_{\chi_{flood}}$
  Compute $\mathbf{u}_a = \mathbf{A}^\top \cdot \mathbf{r}_a + \mathbf{e}_{u,a} \pmod{\mathcal{Q}}$
  Compute $v_a = \langle \mathbf{pk}_{global}, \mathbf{r}_a \rangle + e_{v,a} + \mathbf{s}_a \pmod{\mathcal{Q}}$
  Broadcast $\mathbf{C}_a^{(i)} = (\mathbf{u}_a, v_a)$ to all nodes
end for
Receive $\mathbf{C}_a^{(j)}$ from other nodes
Compute $\mathbf{CG}_{u,a} = \sum_{j=1}^{N} \mathbf{u}_a^{(j)} \pmod{\mathcal{Q}}$
Compute $CG_{v,a} = \sum_{j=1}^{N} v_a^{(j)} \pmod{\mathcal{Q}}$
for $(a, b) \in \{1, \ldots, n\}^2$ AND $k \in \{1, \ldots, r\}$ **do**
  Compute $\mathbf{t}_u = \mathbf{CG}_{u,a} \cdot \mathbf{s}_b \cdot \hat{q}_k \pmod{q_k}$
  Compute $t_v = CG_{v,a} \cdot \mathbf{s}_b \cdot \hat{q}_k \pmod{q_k}$
  Sample flood error vector $\mathbf{e}_{u,flood} \in \mathcal{R}_{\chi_{flood}}^n$
  Sample flood error polynomial $e_{v,flood} \in \mathcal{R}_{\chi_{flood}}$
  Compute $\mathbf{sh_u}_{(a,b),k} = \mathbf{t}_u + \mathbf{e}_{u,flood} \pmod{q_k}$
  Compute $sh_v_{(a,b),k} = t_v + e_{v,flood} \pmod{q_k}$
end for
Broadcast $rlk_share^{(i)} = (\mathbf{sh_u}^{(i)}, sh_v^{(i)})$
for $(a, b) \in \{1, \ldots, n\}^2$ AND $k \in \{1, \ldots, r\}$ **do**
  Compute $\mathbf{rlk_u}_{(a,b),k} = \sum_{j=1}^{N} \mathbf{sh_u}_{(a,b),k}^{(j)} \pmod{q_k}$
  Compute $rlk_v_{(a,b),k} = \sum_{j=1}^{N} sh_v_{(a,b),k}^{(j)} \pmod{q_k}$
  Set Global RLK $RLK_{global} = \{(\mathbf{rlk_u}_{(a,b),k}, rlk_v_{(a,b),k})\}$
end for

3) **RLKGen:** Algorithm 3 is executed by DCN to securely generate the global relinearization key (RLK) required for dimensionality reduction following homomorphic multiplication. First, each node encryptes its individual secret key fragments $\mathbf{s}_a$ using the global public key $\mathbf{pk}_{global}$, injecting flooding noise with a massive variance to guarantee FuncCPA security. These ciphertexts $\mathbf{C}_a^{(i)}$ then broadcast to the entire network. When receiving ciphertexts from all nodes, they are homomorphically aggregated to form global ciphertexts that encrypt the components of the global secret key. After that, nodes compute the quadratic cross-terms necessary to construct the RLK. To keep the computations within smaller moduli, it utilizes RNS digit decomposition, multiplying the aggregated ciphertexts by the node's own secret key fragment $\mathbf{s}_b$ and the CRT constant $\hat{q}_k$. Then, nodes inject massive flooding noise $(\mathbf{e}_{u,flood}, e_{v,flood})$ once again to completely obfuscate the algebraic traces of the secret key cross-multiplications, yielding RLK shares that are FuncCPA-secure. Finally the system sums the broadcasted shares from all nodes under each RNS limb q_k to assemble the final global RLK.

4) **Encryption:** Algorithm 4 is executed by any DO to encrypt a plaintext message into an evaluable ciphertext using the shared global public key established by the network. First, the DO encodes the message vector $\mathbf{m}_{vec}$ into a polynomial m_{poly} via the Inverse Number Theoretic Transform (iNTT). Next, DO samples a random short vector $\mathbf{r}$ from the uniform short noise distribution to serve as the ephemeral randomness for this encryption. After that, DO samples flooding noise $\mathbf{y}_0$ and y_1 with massive variance. Then, DO computes the masking component of the ciphertext $\mathbf{u}$ and the payload component v. The purpose of multiplying by Δ is to shift the plaintext into the most significant bits of the ciphertext modulus $\mathcal{Q}$, ensuring it is not corrupted by the underlying massive flooding noise. Finally, DO output the resulting ciphertext pair $(\mathbf{u}, v)$.

5) **Partial Decryption:** The decentralized node, which holds a secret key fragment, executes Algorithm 5 to compute its share of the decryption for a given ciphertext , while simultaneously generating a ZKP to assure the network of its honest computation without leaking any underlying secret key information. First, the node computes the inner product of the ciphertext's vector portion $\mathbf{u}$ and its own secret key fragments $\mathbf{s}^{(i)}$ to obtain the raw decryption share. To satisfy FuncCPA security requirements, the node samples ideal flooding noise e_{flood} with a massive variance and adds it to the raw share. Next, the node proceeds to construct the HQ-FS ZKP. The node samples small random mask vectors $\mathbf{y}_s$ and $\mathbf{y}_e$, and computes the ciphertest-bound commitment $\mathbf{T_u}$ and the public-key-bound commitment $\mathbf{T}_A$ modulo the dedicated ZKP baseline prime $K_\mathcal{Q}$. Then, node use Fiat-Shamir heuristic to generate a non-interactive challenge value ch. Finally, the node computes the proof responses $\mathbf{z}_s$ and $\mathbf{z}_e$ by combining the mask vectors with the challenge, its secret key fragment, and the ZKP error term. The node outputs this proof tuple π alongside the masked decryption share $sh^{(i)}$.

6) **Decryption:** Algorithm 6 is executed by DR to recover the message. First, DR uses Breadth-First Search (BFS) by

Algorithm 4 Encryption $(pp, \mathbf{pk}_{global}, \mathbf{m})$

Require: Public parameter pp, global public key $\mathbf{pk}_{global}$, message vector $\mathbf{m}$
Ensure: Ciphertext c
1: Encode massage vector $m_{poly} = \text{iNTT}(\mathbf{m}) \pmod T$
2: Sample vector $\mathbf{r} \in \chi_\eta^m$
3: Sample error vector $\mathbf{y}_0 \in \chi_{flood}^n$
4: Sample error polynomial $y_1 \in \chi_{flood}$
5: Compute $\mathbf{u} = \mathbf{A}^\top \cdot \mathbf{r} + \mathbf{y}_0 \pmod{\mathcal{Q}}$
6: Compute $v = \langle \mathbf{pk}_{global}, \mathbf{r} \rangle + y_1 + \Delta m \pmod{\mathcal{Q}}$
7: Set ciphertext $c = (\mathbf{u}, v)$

Algorithm 5 Partial Decryption $(\mathbf{u}, \mathbf{s}^{(i)}, \mathbf{e}_{zkp}^{(i)}, \mathbf{A}_{zkp})$

Require: Ciphertext vector $\mathbf{u}$, node secret share $\mathbf{s}^{(i)}$, ZKP error $\mathbf{e}_{zkp}^{(i)}$, HQ-FS ZKP public reference matrix $\mathbf{A}_{zkp}$
Ensure: Partial decryption share $sh^{(i)}$, the proof $\pi^{(i)}$
1: Compute $sh_{raw}^{(i)} = \langle \mathbf{u}, \mathbf{s}^{(i)} \rangle$
2: Sample ideal flood noise $e_{flood} \in [-B_{flood}, B_{flood}]$
3: Compute partial decryption share $sh^{(i)} = sh_{raw}^{(i)} + e_{flood}$
4: Sample mask polynomial vector $\mathbf{y}_s \in \mathcal{R}_{\mathcal{U}_\eta}^n$
5: Sample mask polynomial vector $\mathbf{y}_e \in \mathcal{R}_{\mathcal{U}_\eta}^m$
6: Compute $\mathbf{T_u} = \langle \mathbf{u}, \mathbf{y}_s \rangle \pmod{K_\mathcal{Q}}$
7: Compute $\mathbf{T}_A = \mathbf{A}_{zkp} \cdot \mathbf{y}_s + \mathbf{y}_e \pmod{K_\mathcal{Q}}$
8: Get challenge $ch = H(\mathbf{A}_{zkp}, \mathbf{C}, \mathbf{u}, \mathbf{T}_u, \mathbf{T}_A) \to \{0,1\}^d \in \mathcal{R}_\mathcal{Q}$
9: Compute $\mathbf{z}_s = \mathbf{y}_s + ch \cdot \mathbf{s}^{(i)} \pmod{K_\mathcal{Q}}$
10: Compute $\mathbf{z}_e = \mathbf{y}_e + ch \cdot \mathbf{e}_{zkp}^{(i)} \pmod{K_\mathcal{Q}}$
11: Proof $\pi = (ch, \mathbf{z}_s, \mathbf{z}_e, \mathbf{T}_u, \mathbf{T}_A)$

queue to group nodes and perform homomorphic aggregation verification. For each subset, DR aggregates the submitted proof responses $(\mathbf{Z}_s, \mathbf{Z}_e)$, commitments $(\mathbf{T}_u^{agg}, \mathbf{T}_A^{agg})$, challenge-weighted decryption shares $\mathbf{S}^{agg}$, and public key commitments $\mathbf{C}^{agg}$. After that, DR simultaneously check whether the verifying the correctness of the decryption relation, ensuring the secret key binds consistently to the initial registration commitment, and confirming that the noise remains strictly within the safety bounds are pass or not. If all checks pass, the nodes in the subset are deemed honest and added to the valid set $\mathcal{V}$. If all verfication fails, the subset is split into smaller sub-groups and pushed back into the queue for recursive filtering. When a subset is reduced to a single node and fails the checks, the malicious node is implicitly and precisely dropped. After ZKP verification, DR aggregates the decryption shares from all valid honest nodes sh^{agg} and subtracts this sum from the scalar portion v of the ciphertext, yielding the phase ph_{rns} with the secret key layer neutralized. Then DR employs the CRT to convert the RNS representation back into a large integer polynomial. After symmetrically centering the coefficients, DR divides them by the scaling factor Δ and rounds to the nearest integer. Finally, using the Number Theoretic Transform, DR extracts the data slots from the polynomial and recovers the final plaintext vector $\mathbf{m}_{vec}$.

7) **Addition:** CSP execute Algorithm 7 to generate the additional ciphertext. First, CSP algebraically adds the masking components $\mathbf{u}$ and the payload components v of two input ciphertexts $c^{(\alpha)}$ and $c^{(\beta)}$ modulo $\mathcal{Q}$ to get the temporary sum $(\mathbf{u}_{tmp}, v_{jtmp})$. Next, to ensure circuit privacy and support

Algorithm 6 Decryption$(c, \{sh^{(i)}, \pi^{(i)}, \mathbf{C}^{(i)}\}_{i \in N})$

Require: Ciphertext c, sharing set $\{(sh^{(i))}, \pi^{(i)}, \mathbf{C}^{(i)}\}_{i \in N}$
Ensure: Plaintext m_{vec}
1: Initialize queue $\ell \leftarrow \{\mathcal{L}\}$
2: Initialize valid node set $\mathcal{V} \leftarrow \emptyset$
3: **while** ℓ is not empty **do**
4: Pop a subset $\mathcal{S}$ from ℓ
5: Compute $\mathbf{Z}_s = \sum_{i \in \mathcal{S}} \mathbf{z}_s^{(i)} \pmod{K_\mathcal{Q}}$
6: Compute $\mathbf{Z}_e = \sum_{i \in \mathcal{S}} \mathbf{z}_e^{(i)} \pmod{K_\mathcal{Q}}$
7: Compute $\mathbf{T}_u^{agg} = \sum_{i \in \mathcal{S}} \mathbf{T}_u^{(i)} \pmod{K_\mathcal{Q}}$
8: Compute $\mathbf{T}_A^{agg} = \sum_{i \in \mathcal{S}} \mathbf{T}_A^{(i)} \pmod{K_\mathcal{Q}}$
9: Compute $\mathbf{S}^{agg} = \sum_{i \in \mathcal{S}} ch^{(i)} \cdot sh^{(i)} \pmod{K_\mathcal{Q}}$
10: Compute $\mathbf{C}^{agg} = \sum_{i \in \mathcal{S}} ch^{(i)} \cdot \mathbf{C}^{(i)} \pmod{K_\mathcal{Q}}$
11: $b_{dec} \leftarrow (\langle \mathbf{u}, \mathbf{Z}_s \rangle - \mathbf{S}^{agg} == \mathbf{T}_u^{agg} \pmod{K_\mathcal{Q}})$
12: $b_{key} \leftarrow (\mathbf{A}_{zkp} \cdot \mathbf{Z}_s + \mathbf{Z}_e - \mathbf{C}^{agg} == \mathbf{T}_A^{agg} \pmod{K_\mathcal{Q}})$
13: $b_{norm} \leftarrow (\|\mathbf{Z}_s\|_\infty \leq K_\mathcal{Q}/4)$
14: **if** b_{dec} AND b_{key} AND b_{norm} **then**
15: $\mathcal{V} \leftarrow \mathcal{V} \cup \mathcal{S}$
16: **else if** $|\mathcal{S}| > 1$ **then**
17: Split $\mathcal{S}$ into N sub-groups: $\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_N$
18: Push $\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_N$ into ℓ
19: **end if**{If $|\mathcal{S}| == 1$ and checks fail, the malicious node is implicitly dropped}
20: **end while**
21: Aggregate valid shares: $sh^{agg} = \sum_{i \in \mathcal{V}} sh^{(i)} \pmod{\mathcal{Q}}$
22: Compute phase: $ph_{rns} = v - sh^{agg} \pmod{\mathcal{Q}}$
23: Convert RNS to big integer: $coeffs_{big} = \text{CRT}(ph_{rns})$
24: Center coefficients $coeffs_{big}$ into symmetric interval $[-\mathcal{Q}/2, \mathcal{Q}/2]$
25: Scale and round to remove noises: $m_{poly} = \lfloor \frac{coeffs_{big} + \Delta/2}{\Delta} \rfloor \pmod T$
26: Extract plaintext slots via NTT: $\mathbf{m}_{vec} = \text{NTT}(m_{poly}) \pmod T$

Algorithm 7 Addition$(c^{(\alpha)}, c^{(\beta)})$

Require: Ciphertext $c^{(\alpha)}$ from $U^{(\alpha)}$, Ciphertext $c^{(\beta)}$ from $U^{(\beta)}$
Ensure: Ciphertext c_{add}
1: Compute $\mathbf{u}_{tmp} = \mathbf{u}^{(\alpha)} + \mathbf{u}^{(\beta)} \pmod{\mathcal{Q}}$
2: Compute $v_{tmp} = v^{(\alpha)} + v^{(\beta)} \pmod{\mathcal{Q}}$
3: Get $c_{zero} = (\mathbf{u}_{zero}, v_{zero}) \leftarrow \mathsf{Encryption}(pp, \mathbf{pk}_{global}, 0)$
4: Compute $\mathbf{u}_{add} = \mathbf{u}_{tmp} + \mathbf{u}_{zero} \pmod{\mathcal{Q}}$
5: Compute $v_{add} = v_{tmp} + v_{zero} \pmod{\mathcal{Q}}$
6: Addition ciphertext $c_{add} = (\mathbf{u}_{add}, v_{add})$

the system's overall FuncCPA security, we introduce a re-randomization mechanism. CSP calls Algorithm 4 the value 0 under the global key, generating a "zero ciphertext" that contains fresh flooding noise. Finally, CSP superimposes this zero-ciphertext onto the previous temporary result, completely refreshing the noise distribution within the ciphertext and erasing the deterministic algebraic traces left by directly adding the two ciphertexts.

8) **Multiplication:** CSP execute Algorithm 8 to generate the multiplication ciphertext. First, CSP computes the scalar multiplication term v_{final} and the linear corss-terms $\mathbf{u}_{final}$. When the $\mathbf{u}$ vectors of two ciphertexts are multiplied, it generates quadratic terms w_{rns} that depend on the square of the secret key, causing the ciphertext dimensions to expand. To restore the ciphertext to its original linear structure, we use RNS digit decomposition. For each quadratic term,

Algorithm 8 Multiplication($rlk, c^{(\alpha)}, c^{(\beta)}$)

Require: Ciphertext $c^{(\alpha)}$ from $U^{(\alpha)}$, Ciphertext $c^{(\beta)}$ from $U^{(\beta)}$
Ensure: Ciphertext c_{mul}
Compute the scalar-term $v_{final} = v^{(\alpha)} \cdot v^{(\beta)} \pmod{\mathcal{Q}}$
Get the cross-term $\mathbf{u}_{final} = (\mathbf{u}^{(\alpha)} v^{(\beta)} + \mathbf{u}^{(\beta)} v^{(\alpha)}) \pmod{\mathcal{Q}}$
for $(a, b) \in (n, n)$ **do**
  Compute $w_{rns} = \mathbf{u}_a^{(\alpha)} \cdot \mathbf{u}_b^{(\beta)}$
  for $i \in r$ **do**
    Compute $dig = (w_{rns}[i] \cdot \hat{q}_i^{-1}) \pmod{q_i}$
    for $k \in r$ **do**
      if $k \neq i$ **then**
        $dig = dig \pmod{q_k}$
      end if
      $(key_u, key_v) = rlk_{(a,b),k}$
      Compute $\mathbf{u}_{final} = \mathbf{u}_{final}[t] + (dig \cdot key_u[t])$
      Compute $v_{final} = v_{final} + (dig \cdot key_v)$
    end for
  end for
end for
Set $c_{zero} = (\mathbf{u}_{zero}, v_{zero}) \leftarrow \mathsf{Encryption}(pp, \mathbf{pk}_{global}, 0)$
Compute $\mathbf{u}_{mul} = \mathbf{u}_{final} + \mathbf{u}_{zero}$
Compute $v_{mul} = v_{final} + v_{zero}$
Multiplication ciphertext $c_{mul} = (\mathbf{u}_{mul}, v_{mul})$

CSP decomposes it across the RNS prime bases to extract a digit dig. Then, dig is base-extended to all RNS limbs and homomorphically accumulated by multiplying it with the pre-generated global RLK. Finally, to ensure circuit privacy, CSP follows Algorithm 7 to encrypt 0 and generate a zero-ciphertext carry with fresh noise. This zero-ciphertext is superimposed on the reduced result, yielding the final secure multiplication ciphertext c_{mul}.

IV. Security Analysis

This section discusses the correctness and formal security of the proposed protocol.

A. Correctness

The correctness of our scheme is ensured in three steps: 1. encryption and the global key, 2. threshold decryption and fault-tolerance verification, and 3. homomorphic operations and circuit privacy, which are discussed in detail below.

1) Encryption and the Global Key: Our scheme adopts an RNS-based BFV/BGV variant. The global key is a superposition of the public key shares from DCN, and its algebraic structure guarantees a correct encryption-decryption mapping.

a) Global Key Structure: According to Algorithm 2, the public key of node i is $\mathbf{pk}^{(i)} = \mathbf{A}\mathbf{s}^{(i)} + \mathbf{e}^{(i)} \pmod{\mathcal{Q}}$. The global public key is the sum of all individual public keys

$$\begin{aligned} \mathbf{pk}_{global} &= \sum_{i=1}^{N} \mathbf{pk}^{(i)} = \mathbf{A} \sum_{i=1}^{N} \mathbf{s}^{(i)} + \sum_{i=1}^{N} \mathbf{e}^{(i)} \\ &= \mathbf{A}\mathbf{s}_{global} + \mathbf{e}_{global} \pmod{Q} \end{aligned} \tag{9}$$

where $\mathbf{s}_{global}$ is the global unknown secret key, and $\mathbf{e}_{global}$ is the global initial error.

b) Correctness of the Encryption Process: According to Algorithm 4, DO encodes and encrypts the plaintext m using $\mathbf{pk}_{global}$ to generate the ciphertext $c = (\mathbf{u} = \mathbf{A}^\top \mathbf{r} + \mathbf{y}_0, v = \langle \mathbf{pk}_{global}, \mathbf{r} \rangle + y_1 + \Delta m \pmod{\mathcal{Q}})$. We can expand v by substituting $\mathbf{pk}_{global}$ into the equation to obtain

$$\begin{aligned} v &= \langle \mathbf{A}\mathbf{s}_{global} + \mathbf{e}_{global}, \mathbf{r} \rangle + y_1 + \Delta m \pmod{\mathcal{Q}} \\ &= \mathbf{r}^\top \mathbf{A}\mathbf{s}_{global} + \langle \mathbf{e}_{global}, \mathbf{r} \rangle + y_1 + \Delta m \pmod{\mathcal{Q}} \end{aligned} \tag{10}$$

During decryption, the phase of the ciphertext is computed as $v - \langle \mathbf{u}, \mathbf{s}_{global} \rangle$, which yields

$$\begin{aligned} & v - \langle \mathbf{u}, \mathbf{s}_{global} \rangle \\ &= \Delta m + \langle \mathbf{e}_{global}, \mathbf{r} \rangle + y_1 - \langle \mathbf{y}_0, \mathbf{s}_{global} \rangle \pmod{\mathcal{Q}}. \end{aligned} \tag{11}$$

Let the encryption noise be defined as $\text{noise}_{enc} = \langle \mathbf{e}_{global}, \mathbf{r} \rangle + y_1 - \langle \mathbf{y}_0, \mathbf{s}_{global} \rangle$. The decryption phase is then strictly equal to $\Delta m + \text{noise}_{enc} \pmod{\mathcal{Q}}$. As long as the infinity norm of noise_{enc} is less than $\frac{\Delta}{2}$, the plaintext m can be correctly recovered by dividing by Δ and rounding to the nearest integer.

2) Threshold Decryption and Fault-Global Key: Algorithm 5 and Algorithm 6 ensure FuncCPA security and resistance against malicious tampering during the decryption process through distributed computation and ZKP.

a) Partial Decryption and Flooding Noise: The decryption share submitted by node i is $sh^{(i)} = \langle \mathbf{u}, \mathbf{s}^{(i)} \rangle + e_{flood}^{(i)}$. The aggregated decryption share sh^{agg} from all honest nodes in the valid set $\mathcal{V}$ is evaluated as $sh^{agg} = \sum_{i \in \mathcal{V}} sh^{(i)} = \langle \mathbf{u}, \mathbf{s}_{global} \rangle + \sum_{i \in \mathcal{V}} e_{flood}^{(i)} \pmod{\mathcal{Q}}$. Substituting this into the decryption phase calculation results in $ph_{rns} = v - sh^{agg} = \Delta m + \text{noise}_{enc} - \sum_{i \in \mathcal{V}} e_{flood}^{(i)} \pmod{\mathcal{Q}}$. The massive flooding noise $\sum e_{flood}^{(i)}$ completely masks $sh_{raw}^{(i)}$, thereby ensuring FuncCPA security. As long as the system parameters are set such that the total noise $\|\text{noise}_{enc} - \sum e_{flood}\|_\infty \leq \frac{\Delta}{2}$, the scaling and rounding operation $\lfloor \frac{coeffs_{big} + \frac{\Delta}{2}}{\Delta} \rfloor$ in Algorithm 6 removes all noise and successfully extracts the slots $\mathbf{m}_{rec}$.

b) Correctness of HQ-FS ZKP Verification: To prevent nodes from cheating on $sh^{(i)}$, Algorithm 6 performs homomorphic aggregation verification under the small baseline modulus $K_\mathcal{Q}$ through two primary checks.
Firstly, the Secret Key Consistency Check (b_{key}) evaluates the expression $\mathbf{A}_{zkp}\mathbf{Z}_s + \mathbf{Z}_e - \mathbf{C}^{agg} \pmod{K_\mathcal{Q}}$. By substituting $\mathbf{Z}_s = \sum(\mathbf{y}_s^{(i)} + ch^{(i)}\mathbf{s}^{(i)})$, $\mathbf{Z}_e = \sum(\mathbf{y}_e^{(i)} + ch^{(i)}\mathbf{e}_{zkp}^{(i)})$, and $\mathbf{C}^{(i)} = \mathbf{A}_{zkp}\mathbf{s}^{(i)} + \mathbf{e}_{zkp}^{(i)}$, we obtain

$$\begin{aligned} & \sum(\mathbf{A}_{zkp}\mathbf{y}_s^{(i)} + \mathbf{y}_e^{(i)}) + \sum ch^{(i)}(\mathbf{A}_{zkp}\mathbf{s}^{(i)} + \mathbf{e}_{zkp}^{(i)}) - \sum ch^{(i)}\mathbf{C}^{(i)} \\ &= \sum \mathbf{T}_A^{(i)} = \mathbf{T}_A^{agg} \pmod{K_\mathcal{Q}}. \end{aligned} \tag{12}$$

This equality strictly holds, which proves that the $\mathbf{s}^{(i)}$ used by the node is strictly bound to the initially registered commitment $\mathbf{C}^{(i)}$. Secondly, the Decryption Relation Check (b_{dec}) evaluates the expression $\langle \mathbf{u}, \mathbf{Z}_s \rangle - \mathbf{S}^{agg} \pmod{K_\mathcal{Q}}$. By sub-

stituting $\mathbf{Z}_s$ and $\mathbf{S}^{agg} = \sum ch^{(i)} sh^{(i)} = \sum ch^{(i)}(\langle \mathbf{u}, \mathbf{s}^{(i)} \rangle + e^{(i)}_{flood})$, the expression becomes

$$\begin{aligned} &\sum \langle \mathbf{u}, \mathbf{y}^{(i)}_s + ch^{(i)} \mathbf{s}^{(i)} \rangle - \sum ch^{(i)} \langle \mathbf{u}, \mathbf{s}^{(i)} \rangle - \sum ch^{(i)} e^{(i)}_{flood} \\ &\quad = \sum \langle \mathbf{u}, \mathbf{y}^{(i)}_s \rangle - \sum ch^{(i)} e^{(i)}_{flood} \\ &\quad = \mathbf{T}^{agg}_u - \sum ch^{(i)} e^{(i)}_{flood} \pmod{K_{\mathcal{Q}}}. \end{aligned} \tag{13}$$

For this to equal $\mathbf{T}^{agg}_u \pmod{K_{\mathcal{Q}}}$, the implicit correctness condition of the system requires that the flooding noise $e^{(i)}_{flood}$ must be a multiple of $K_{\mathcal{Q}}$. Under this condition, a malicious node cannot tamper with $sh^{(i)}$ and simultaneously pass the verification. The BFS queue can then precisely filter out all malicious shares that fail the b_{dec} or b_{key} checks.

3) Homomorphic Operations and Circuit Privacy: Algorithm 7 and Algorithm 8 define the homomorphic addition and multiplication, enforcing the superposition of a zero-ciphertext at the end of the operations to guarantee circuit privacy.

a) Homomrphic Addition (Algorithm 7): Let the phases corresponding to the input ciphertexts be $ph^{(\alpha)} = \Delta m_\alpha + e_\alpha$ and $ph^{(\beta)} = \Delta m_\beta + e_\beta$. The phase of the temporary ciphertext $(\mathbf{u}_{tmp}, v_{tmp})$ after direct algebraic addition is determined by

$$v_{tmp} = \langle \mathbf{u}_{tmp}, \mathbf{s}_{global} \rangle = \Delta(m_\alpha + m_\beta) + (e_\alpha + e_\beta) \pmod{\mathcal{Q}}.$$

This process preserves the homomorphic property of addition. By superimposing the zero ciphertext $c_{zero} = (\mathbf{u}_{zero}, v_{zero})$, the plaintext value increases by 0, but the encryption introduces entirely new noise. This preserves the algebraic traces between the output/input ciphertexts, ensuring that the output distribution is statistically indistinguishable from a fresh encryption of $(m_\alpha + m_\beta)$, thereby satisfying FuncCPA security.

b) Homomorphic Multiplication (Algorithm 8): Multiplication gnereates a tensor product cross-term $w_{rns} = \mathbf{u}^{(\alpha)}_a \cdot \mathbf{u}^{(\beta)}_b$. This transforms the decryption phase into a non-linear format $\langle \mathbf{u}^{(\alpha)} \otimes \mathbf{u}^{(\beta)}, \mathbf{s}_{global} \otimes \mathbf{s}_{global} \rangle$. Through RNS decomposition, the quadratic term w_{rns} is decomposed into digits $dig = (w_{rns}[i] \cdot \hat{q}_i^{-1}) \pmod{q_i}$. The pre-generated RLK_{global} from Algorithm 3 encapsulates the homomorphic encryption of the secret key products $\mathbf{s}_a \mathbf{s}_b \hat{q}_k$, which is safeguarded by massive flooding noise $\mathbf{e}_{u,flood}$ to protect the key. In Algorithm 8, given the global secret key $\mathbf{s}$, the correct decryption phases of the two input ciphertexts $c^{(\alpha)}$ and $c^{(\beta)}$ satisfy $v^{(\alpha)} - \langle \mathbf{u}^{(\alpha)}, \mathbf{s} \rangle = \Delta m_\alpha + e_\alpha \pmod{\mathcal{Q}}$ and $v^{(\beta)} - \langle \mathbf{u}^{(\beta)}, \mathbf{s} \rangle = \Delta m_\beta + e_\beta \pmod{\mathcal{Q}}$ respectively. Multiplying these two equations and restructuring the scalar, linear, and quadratic terms yields the non-linear decryption equation

$$\begin{aligned} &v^{(\alpha)} v^{(\beta)} - \langle v^{(\beta)} \mathbf{u}^{(\alpha)} + v^{(\alpha)} \mathbf{u}^{(\beta)}, \mathbf{s} \rangle + \sum_{a,b} w_{a,b} (\mathbf{s}_a \mathbf{s}_b) \\ &\qquad = \Delta^2 m_\alpha m_\beta + e_{cross} \pmod{\mathcal{Q}} \end{aligned} \tag{14}$$

where $w_{a,b} = \mathbf{u}^{(\alpha)}_a \mathbf{u}^{(\beta)}_b$ represents the tensor product cross-term of the ciphertexts. To eliminate the quadratic secret key term $\mathbf{s}_a \mathbf{s}_b$ that causes ciphertext dimensionality expansion and to strictly control noise growth, the system employs RNS digit decomposition to precisely parse the quadratic term $w_{a,b}$ along each prime basis into low-weight components $dig_{a,b,i}$, satisfying $\sum_i dig_{a,b,i} \cdot \hat{q}_i \equiv w_{a,b} \pmod{\mathcal{Q}}$. Subsequently, the system introduces the algebraic definition of the global relinearization key RLK which implies a linear encryption relationship expressed as $key_{v,a,b,i} - \langle \mathbf{key}_{u,a,b,i}, \mathbf{s} \rangle = \mathbf{s}_a \mathbf{s}_b \hat{q}_i + e_{rlk,a,b,i} \pmod{\mathcal{Q}}$. Following the algorithmic design, a homomorphic update for dimensionality reduction is executed by computing $\mathbf{u}_{final} = (v^{(\beta)} \mathbf{u}^{(\alpha)} + v^{(\alpha)} \mathbf{u}^{(\beta)}) + \sum_{a,b,i} dig_{a,b,i} \cdot \mathbf{key}_{u,a,b,i}$ and $v_{final} = (v^{(\alpha)} v^{(\beta)}) + \sum_{a,b,i} dig_{a,b,i} \cdot key_{v,a,b,i}$. The new decryption phase of this substitution operation $v_{final} - \langle \mathbf{u}_{final}, \mathbf{s} \rangle$ is algebraically simplified to $\Delta^2 m_\alpha m_\beta + e_{cross} + e_{relin} \pmod{\mathcal{Q}}$. This proves that the non-linear quadratic secret-key term is eliminated and the ciphertext reverts to the linear algebraic structure $\langle \mathbf{u}_{final}, \mathbf{s} \rangle$. However, after dimensionality reduction, the residual computation noise $e_{tmp} = e_{cross} + e_{relin}$ still retains strong deterministic algebraic dependencies on the underlying computation paths and initial inputs. To satisfy the rigorous FuncCPA security model, the system forcibly introduces a zero-ciphertext $c_{zero} = (\mathbf{u}_{zero}, v_{zero})$ sampled from a massive variance flooding distribution at the end of the operation, with its phase satisfying $v_{zero} - \langle \mathbf{u}_{zero}, \mathbf{s} \rangle = e_{flood} \pmod{\mathcal{Q}}$ and $e_{flood} \leftarrow \chi_{flood}$. Through the final ciphertext superposition operation $c_{mul} = c_{tmp} + c_{zero}$, the output phase is transformed into $\Delta^2 m_\alpha m_\beta + (e_{tmp} + e_{flood}) \pmod{\mathcal{Q}}$. Since the system parameters guarantee that the norm of the flooding noise is significantly larger than the internal computation noise $\|e_{flood}\| \gg \|e_{tmp}\|$, the statistical hiding property of the Smudging Lemma $e_{tmp} + e_{flood} \approx_s e_{flood}$ ensures that the deterministic algebraic computation trace e_{tmp} is completely overwhelmed and obliterated statistically by the massive flooding noise. This renders the final output multiplication ciphertext statistically indistinguishable from a fresh encryption generated directly from the public key. Consequently, while maintaining the correctness of homomorphic operations, the system achieves the highest standard of circuit privacy and active security through a rigorous algebraic structure.

B. Formal Security

In this part, we formally analyze the security of our proposed scheme. The analysis will focus on three parts: (1) Unified IND-FuncCPA security under $N-1$ corruptions, (2) Robustness and Zero-Knowledge of HQ-FS ZKP, and (3) Simulation-based security in the real-ideal paradigm.

1) Unified IND-FuncCPA security under $N-1$ corruptions: We consider a static malicious adversary $\mathcal{A}$ that corrupts up to $N-1$ nodes. To capture both the privacy of fresh ciphertexts and the leakage-resilience of homomorphic evaluations and decryption shares, we discussed the Indistinguishability under Functional Chonsen-Plaintext Attack (IND-FuncCPA) game between a Challenger $\mathcal{C}$ and $\mathcal{A}$ in Definition 6.

Theorem 1 (Comprehensive IND-FuncCPA Security)**.** *Assuming the hardness of the* GLWE *problem (Definition 1), the proposed scheme achieves* IND-FuncCPA *security. Specifi-*

cally, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ making at most q queries, the advantage of $\mathcal{A}$ satisfies:

$$\mathrm{Adv}_{\mathcal{A}}^{\text{IND-FuncCPA}}(\lambda) \leq \mathrm{Adv}_{\mathcal{A}}^{\mathsf{GLWE}}(\lambda) + q \cdot \epsilon_{\text{smudge}} \leq \mathrm{negl}(\lambda),$$

where the flooding noise bound $B_{\text{flood}} \geq 2^{\lambda} \cdot \max(|x|)$, guaranteeing that the statistical distance satisfies $\epsilon_{\text{smudge}} \leq 2^{-\lambda}$.

Proof. We demonstrate the security via a hybrid argument.

- **Game 0 (Real Game):** This is the original IND-FuncCPA game. Thus, $Adv_{G_0}(\lambda) = Adv_{\mathcal{A}}^{IND-FuncCPA}(\lambda)$.
- **Game 1 (Simulated Evaluations and shares):** We modify the Evaluation and Partial Decryption oracles. Instead of computing the exact algebraic noise for evaluated ciphertexts and the honest node's raw decryption share, $\mathcal{C}$ returns freshly encrypted evaluated plaintexts and uniformly sampled shares from the massive flooding distribution $\mathcal{R}_{\mathcal{U}_{[-B_{flood},B_{flood}]}}$. By Definition 1, the massive variance of the flooding noise statistically drowns the structural evaluation noise and the secret-dependent raw share. The statistical distance between G_0 and G_1 is bounded by $q \cdot \epsilon_{smudge}$. Thus, $|Adv_{G_0} - Adv_{G_1}| \leq q \cdot \epsilon_{smudge}$.
- **Game 2 (Simulated Public Key and Challenge):** We modify the Setup and Challenge phase. $\mathcal{C}$ replace the honest node's public key share $\mathbf{A}\mathbf{s}^{(honest)} + \mathbf{e}^{(honest)}$ with a uniformly random vector $\mathbf{u}_{pk} \in \mathcal{R}_{\mathcal{Q}}^{m}$. Consequently, $\mathbf{pk}_{global}$ becomes uniformly random. Furthermore, $\mathcal{C}$ replaces the challenge ciphertext ct^* with a uniformly random vector $U_{ct} \in \mathcal{R}_{\mathcal{Q}}^{m+1}$. By Definition 1, G_1 and G_2 are computationally indistinguishable. Thus, $|Adv_{G_1} - Adv_{G_2}| \leq Adv_{\mathcal{A}}^{GLWE}(\lambda)$.

In **Game 2**, both $\mathbf{pk}_{global}$ and ct^* are completely uniformly random and independent of the bit b. Therefore, $\mathcal{A}$'s probability of guessing b correctly is exactly $\frac{1}{2}$, it shows $Adv_{G_2}(\lambda) = 0$. Under Definition 1, $Adv_{\mathcal{A}}^{GLWE}(\lambda)$ is a negligible function. Furthermore, since q is polynomial in λ and $\epsilon_{smudge} \leq 2^{-\lambda}$, the term $q \cdot \epsilon_{smudge}$ is also negligible. We can prove the theorem by summing these bounds yields $Adv_{G_0}(\lambda) \leq \mathrm{negl}(\lambda)$. ■

2) Robustness and Zero-Knowledge of HQ-FS ZKP: This section provide security Robustness and Zero-Knowledge.

Theorem 2 (Completeness and Special Soundness)**.** *The HQ-FS ZKP protocol satisfies perfect completeness and computational special soundness, ensuring that maliciously crafted decryption shares are rejected with overwhelming probability.*

Proof. To demonstrate the protocol's resilience against maliciously crafted noise, we prove its completeness and special soundness as follows.

- **Completeness:** At honest node computes $\mathbf{z}_s = \mathbf{y}_s + ch \cdot \mathbf{s}^{(i)}$ and $\mathbf{z}_e = \mathbf{y}_e + ch \cdot \mathbf{e}_{zkp}^{(i)}$. During the BFS aggregation by DR, the aggregated values are exactly the sum of individual honest values. Evaluating $\langle \mathbf{u}, \mathbf{Z}_s \rangle - \mathbf{S}^{agg}$ mathematically yields $\sum \langle \mathbf{u}, \mathbf{y}_s^{(i)} \rangle = \mathbf{T}^{agg}$. Similarly, $\mathbf{A}_{zkp} \cdot \mathbf{Z}_s + \mathbf{Z}_e - \mathbf{C}^{agg} = \mathbf{T}^{agg}$. Hence, an honest aggregated proof always passes verification.
- **Special Soundness:** Assume $\mathcal{A}$ can produce two valid responses $(\mathbf{z}_s, \mathbf{z}_e)$ and $(\mathbf{z}'_s, \mathbf{z}'_e)$ for the same commitments but distinct challenge $ch \neq ch'$. An extractor can recover the secret $\mathbf{s}^{(i)} = (\mathbf{z})_s - \mathbf{z}'_s)(ch - ch')^{-1} \pmod{K_{\mathcal{Q}}}$. Since finding a collision in the cryptographic hash function H is computationally infeasible, $\mathcal{A}$ can not forge a valid proof without knowing $\mathbf{s}^{(i)}$. Moreover, the strict infinity norm check ($\|\mathbf{Z}_s\|_{\infty} \leq \frac{K_{\mathcal{Q}}}{4}$) mathematically prevents the injection of malicious flooding founds.

Hence, it satisfies completeness and special soundness. ■

Theorem 3 (Honest-Verifier Zero-Knowledge)**.** *The HQ-FS ZKP protocol is Honest-Verifier Zero-Knowledge, leaking no information regarding the secret key $\mathbf{s}^{(i)}$.*

Proof. A polynomial-time simulator $\mathcal{S}$ samples a ransom challenge ch and random responses $(\mathbf{z}_s, \mathbf{z}_e)$. It then back-compute the commitments $\mathbf{T}_u = \langle \mathbf{u}, \mathbf{z}_s \rangle - ch \cdot sh_{raw}^{(i)}$ and $\mathbf{T}_A = \mathbf{A}_{zkp} \cdot \mathbf{z}_s + \mathbf{z}_e - ch \cdot \mathbf{C}^{(i)}$. Bacause the true mask vectors $\mathbf{y}_s$ and $\mathbf{y}_e$ are sampled from $\mathcal{R}_{\mathcal{U}_\eta}$ in the real protocol, the distribution of the simulated transcript is identical to an honest prover's transcript. ■

C. Simulation-Based Security in the Real-Ideal Paradigm

This section discusses simulation based security of the proposed protocol.

Theorem 4 (Simulation-Based Security)**.** *The proposed decentralized protocols securely realize the ideal functionality $\mathcal{F}_{FHE}$ in the presence of malicious adversaries corrupting up to $N-1$ nodes, assuming the hardness of the GLWE problem and the random oracle model for the Fiat-Shamir heuristic.*

Proof. We construct a PPT simulator $\mathcal{S}$ interacting $\mathcal{A}$.

- **Simulating KeyGen:** For an honest node i, $\mathcal{S}$ samples a random vector $\mathbf{pk}^{(i)}$. Under GLWE, $\mathcal{A}$ cannot distinguish this from a real public key share. $\mathcal{S}$ simulates the commitment $\mathbf{C}^{(i)}$ based on Theorem 3.
- **Simulating RLKGen:** $\mathcal{S}$ relies on IND-CPA security to simulate the broadcasted $\mathbf{C}_a^{(i)}$. In the second round, $\mathcal{S}$ invokes Lemma 1, sampling uniformly from χ_{flood} to simulate the cross-term shares $\mathbf{sh_u}^{(i)}$ and $sh_v^{(i)}$. The massive variance statistically hides the absence of the correct cross-terms.
- **Simulating Decryption:** When $\mathcal{F}_{FHE}$ outputs $\mathbf{m}_{vec}$, $\mathcal{S}$ algebraically reverse-engineers the required sum of the honest share using the corrupted shares and $\mathbf{m}_{vec}$ to form the correct phase ph_{rns}. Once the required $sh^{(i)}$ is determined, $\mathcal{S}$ uses the HVZK simulator to generate a valid proof tuple $\pi^{(i)}$ that perfectly validates the share without knowing $\mathbf{s}^{(i)}$.

The simulated view is computationally indistinguishable or statistically close to the real execution. Thus, the scheme securely relized $\mathcal{F}_{FHE}$. ■

TABLE II
FEATURE COMPARISON

Scheme	Hardness Assumption	Adversary Model	Decryption Secret Key Leakage Prevention	Active Attack Verification	Circuit Privacy & Anti-Algebraic Tracing	Resistance against CPA^D
[4]	LWE	Static Malicious	Noise Flooding	Reed-Solomon Error Correction	Refresh via Switch-n-Squash	Weak
[13]	RLWE	Semi-Honest	-	Verification of homomorphic computation correctness or Byzantine aggregation	-	Weak
[19]	RLWE	Active Malicious	ZKP	Lattice-based ZKP	Standard Homomorphic & ZKP restrictions	Strong
[21]	LWE	Active Malicious	Smudging noise	NIZK+Commitments	GSW matrix properties and Smudging noise	Moderate
Our Work	GLWE	Active Malicious	Ideal Massive Variance Smudging	HQ-FS ZKP+BFS batch filtering	Injection of Zero-ciphertext	Strong

V. PERFORMANCE

We compare our scheme with four related works [4], [13], [21], [19] in terms of security features (Table II) and computational cost (Table III). Table II considers six aspects: hardness assumption, adversary model, prevention of decryption secret key leakage, active attack verification, circuit privacy and anti-algebraic tracing, and resistance to CPA^D attacks. Table III gives the proof generation and verification time per node, as well as the overall latency of the active defense mechanism.

Our scheme is based on GLWE and tolerates active malicious adversaries. It uses ideal massive variance smudging to prevent secret key leakage, while HQ-FS ZKPs and BFS batch filtering are used to verify active attacks. A zero-ciphertext is also introduced to provide circuit privacy and anti-algebraic tracing. Our scheme provides strong resistance against CPA^D attacks. The proof generation time, verification time, and active defense latency are 1.176 ms, 0.968 ms, and 642.76 ms, respectively. We next compare the four schemes with ours in terms of security and computational cost.

The scheme in [13] is based on RLWE and considers semi-honest parties. It verifies active attacks by checking the correctness of homomorphic computation or through Byzantine aggregation. However, it does not provide mechanisms for decryption secret key leakage prevention or circuit privacy, and its resistance against CPA^D attacks is weak. Its active defense latency is 2857.98 ms, which is $4.45\times$ that of our scheme. Therefore, our scheme provides stronger security with lower active defense cost.

The scheme in [4] is based on LWE and tolerates static malicious adversaries. It uses noise flooding to prevent secret key leakage and Reed–Solomon error correction for active attack verification, and ciphertexts are refreshed using Switch-n-Squash. Its active defense latency is 277.513 ms, which is lower than that of our scheme. However, [4] only tolerates static malicious adversaries and provides weak resistance against CPA^D attacks, whereas our scheme tolerates active malicious adversaries and provides strong resistance against these attacks.

The scheme in [21] is based on LWE and, similar to ours, tolerates active malicious adversaries. It uses smudging noise for decryption secret key leakage prevention and NIZK proofs with commitments to verify active attacks. For circuit privacy, it relies on GSW matrix properties together with smudging noise, and it provides moderate resistance against CPA^D attacks. Its computational cost is higher in all three metrics. The proof generation time, verification time, and active defense latency are 3.73 ms, 2.203 ms, and 5975.068 ms, respectively. Compared with this scheme, our scheme reduces these costs by 68.5%, 56.1%, and 89.2%, respectively. Our scheme therefore provides stronger CPA^D resistance with substantially lower computational cost.

TABLE III
PERFORMANCE COMPARISON (IN *ms*)

Scheme	Proof Generation (each node)	Proof Verification (each node)	Active Defense Latency
[4]	-	-	277.513
[13]	-	-	2857.98
[19]	1.72	0.122	2302.679
[21]	3.73	2.203	5975.068
Our Work	1.176	0.968	642.76

The scheme in [19] is based on RLWE. It uses ZKPs to prevent secret key leakage and lattice-based ZKPs for active attack verification. Circuit privacy is achieved through standard homomorphic operations combined with ZKP restrictions. Like our scheme, it tolerates active malicious adversaries and provides strong resistance against CPA^D attacks, making it the closest prior work to ours in terms of security. Its per-node verification time of 0.122 ms is lower than that of our scheme (0.968 ms). However, its proof generation time is 1.72 ms, or 46.3% higher than ours, while its active defense latency is 2302.679 ms, which is $3.58\times$ that of our scheme. Although the two schemes provide a similar level of security, [19] has a considerably higher overall defense cost.

Overall, the comparison shows a trade-off between security and computational cost among the existing schemes. The scheme in [4] has lower active defense latency but provides weaker security, as it only tolerates static malicious adversaries

and offers weak resistance against CPA^D attacks. The other three schemes provide comparable or weaker security with higher costs. Among the schemes that tolerate active malicious adversaries, our scheme has the lowest proof generation time and active defense latency, while also providing strong resistance against CPA^D attacks.

VI. Conclusion

This paper presents an actively secure and circuit-privacy-preserving threshold multi-party FHE system, addressing the high verification overhead and vulnerability to algebraic tracing attacks (such as $\mathrm{CPA^D}$) prevalent in existing schemes under the malicious model. First, we design the HQ-FS ZKP mechanism, restricted to a small baseline modulus $K_{\mathcal{Q}}$, and integrate it with a Breadth-First Search queue for batch verification. This approach significantly reduces the computational and communication overhead associated with standard ZKPs while precisely isolating malicious nodes. Second, to satisfy the rigorous $\mathrm{FuncCPA}$ security model, we introduce the zero-ciphertext superposition technique. By injecting high-variance flooding noise at the terminal stage of homomorphic operations, this mechanism thoroughly truncates the deterministic algebraic traces left by dimensionality reduction and homomorphic evaluations, providing an efficient defense against $\mathrm{CPA^D}$ attacks without costly bootstrapping. Comprehensive correctness and security analyses show that our system offers breakthrough advantages in active security verification latency and key-recovery attack resistance, while maintaining the compact structure of the RNS-GLWE architecture. Future work will further optimize initial noise budget consumption and explore practical deployments in dynamic decentralized networks and privacy-preserving federated learning scenarios.

References

[1] G. Asharov, A. Jain, A. López-Alt, E. Tromer, V. Vaikuntanathan, and D. Wichs, "Multiparty computation with low communication, computation and interaction via threshold FHE," in *Advances in Cryptology – EUROCRYPT 2012*, pp. 483–501, Springer, 2012.

[2] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai, "Threshold cryptosystems from threshold fully homomorphic encryption," in *Advances in Cryptology – CRYPTO 2018*, pp. 565–596, Springer, 2018.

[3] C. Mouchet, J. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *Proceedings on Privacy Enhancing Technologies*, vol. 2021, no. 4, pp. 291–311, 2021.

[4] M. Dahl, D. Demmler, S. El Kazdadi, A. Meyre, J.-B. Orfila, D. Rotaru, N. P. Smart, S. Tap, and M. Walter, "Noah's ark: Efficient threshold-fhe using noise flooding," in *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pp. 35–46, 2023.

[5] S. Chowdhury, S. Sinha, A. Singh, S. Mishra, C. Chaudhary, S. Patranabis, P. Mukherjee, A. Chatterjee, and D. Mukhopadhyay, "Efficient threshold fhe for privacy-preserving applications," *Cryptology ePrint Archive*, 2022.

[6] K. Boudgoust and P. Scholl, "Simple threshold (fully homomorphic) encryption from LWE with polynomial modulus," in *Advances in Cryptology – ASIACRYPT 2023*, pp. 371–404, Springer, 2023.

[7] A. Passelègue and D. Stehlé, "Low communication threshold fully homomorphic encryption," in *Advances in Cryptology – ASIACRYPT 2024*, pp. 297–329, Springer, 2024.

[8] Y. Chang and S. Li, "Arbitrary-threshold fully homomorphic encryption with lower complexity," in *34th USENIX Security Symposium*, pp. 8403–8422, 2025.

[9] W. Kim, C. Lee, J. Lee, A. Passelègue, and D. Stehlé, "Asynchronous Shamir-based threshold FHE: Smaller moduli and strong simulation security," in *Advances in Cryptology – CRYPTO 2026*, 2026. Cryptology ePrint Archive, Paper 2026/973.

[10] B. Li and D. Micciancio, "On the security of homomorphic encryption on approximate numbers," in *Advances in Cryptology – EUROCRYPT 2021*, Springer, 2021.

[11] J. H. Cheon, H. Choe, A. Passelègue, D. Stehlé, and E. Suvanto, "Attacks against the IND-CPA^D security of exact FHE schemes," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024.

[12] M. Checri, R. Sirdey, A. Boudguiga, and J.-P. Bultel, "On the practical CPAD security of "exact" and threshold FHE schemes and libraries," in *Advances in Cryptology – CRYPTO 2024*, Springer, 2024.

[13] A.-A. Bendoukha, R. Sirdey, A. Boudguiga, and N. Kaaniche, "Unveiling the (in) security of threshold fhe-based federated learning: The practical impact of recent cpad attacks," in *2025 IEEE 38th Computer Security Foundations Symposium (CSF)*, pp. 425–440, IEEE, 2025.

[14] A. Akavia, C. Gentry, S. Halevi, and M. Vald, "Achievable cca2 relaxation for homomorphic encryption," *Journal of Cryptology*, vol. 38, no. 1, p. 5, 2025.

[15] L. Ducas and D. Stehlé, "Sanitization of FHE ciphertexts," in *Advances in Cryptology – EUROCRYPT 2016*, Springer, 2016.

[16] F. Bourse, R. D. Pino, M. Minelli, and H. Wee, "FHE circuit privacy almost for free," in *Advances in Cryptology – CRYPTO 2016*, Springer, 2016.

[17] V. Lyubashevsky, "Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures," in *Advances in Cryptology – ASIACRYPT 2009*, pp. 598–616, Springer, 2009.

[18] V. Lyubashevsky, N. K. Nguyen, and M. Plançon, "Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general," in *Advances in Cryptology – CRYPTO 2022*, pp. 71–101, Springer, 2022.

[19] S. Chatel, C. Mouchet, A. U. Sahin, A. Pyrgelis, C. Troncoso, and J.-P. Hubaux, "Pelta-shielding multiparty-fhe against malicious adversaries," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 711–725, 2023.

[20] I. Hwang, H. Lee, J. Seo, and Y. Song, "Practical zero-knowledge PIOP for maliciously secure multiparty homomorphic encryption," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 4049–4063, 2025.

[21] Y. Xie, R. Huang, and J. Qiu, "Verifiable threshold multi-party fully homomorphic encryption from share resharing," *Applied Sciences*, vol. 15, no. 9, p. 4745, 2025.

[22] E.-S. Z. Ting-Yu Chen, Arijit Karati and C.-I. Fan, "A maliciously secure and fully decentralized threshold fhe scheme with native rns acceleration," 2026.

[23] G. Zyskind, D. Zarchy, M. Leibovich, and C. Peikert, "High-throughput universally composable threshold FHE decryption," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2339–2353, 2025.

[24] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter, S. Lokam, D. Micciancio, D. Moody, T. Morrison, A. Sahai, and V. Vaikuntanathan, "Homomorphic encryption security standard," tech. rep., HomomorphicEncryption.org, Toronto, Canada, Nov. 2018.